COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure & Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	Aug 6, 2026	**DATE SUBMITTED:**	Aug 10, 2026
NAME:	Bal, Kevin Ranz J.	**INSTRUCTOR:**	Engr Jimlord Quejado

Hands-on Activity No.5 Queues

1. PROCEDURE OUTPUT

What procedures did you perform?

PROCEDURE:

Procedure that performed:

ILO A

Task Title & Task Objective

PART A: The Queue ADT
A queue is a list from which items are deleted from one end (front) and into which items are inserted at the other end (rear, or back). It is like a line of people waiting to purchase tickets. Queue is referred to as a first-in- first-out (FIFO) data structure. The first item inserted into a queue is the first item to leave. Queues have many applications in computer systems. Any application where a group of items is waiting to use a shared resource will use a queue.e.g., jobs in a single processor computer, print spooling, information packets in computer networks

Screenshot of the code	

HoA 5.1 > ILO A > iloa.cpp > ...

```cpp
#include <iostream>
#include <queue>

// function declarations
void display(std::queue<char> copyQ);

int main() {

    // create queue object
    std::queue<char> myQ;

    myQ.push('K');
    myQ.push('R');
    myQ.push('B');

    display(myQ);
    std::cout << "current front is: " << myQ.front() << std::endl;
    std::cout << "current back is: " << myQ.back() << std::endl;
    display(myQ);

    // use the dequeue operation
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);
    myQ.pop();

    // check the queue if it is empty
    std::cout << "Is the queue empty? " << myQ.empty() << std::endl;
    return 0;
}

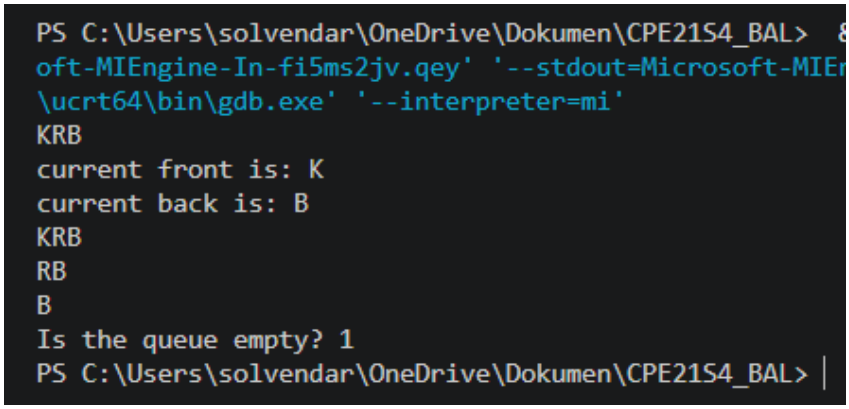
    // note only use the member functions of the queue STL
    void display(std::queue<char> copyQ) {

    // loop until empty using the copy directly
    while (!copyQ.empty()) {

        // display the front
        std::cout << copyQ.front();

        // dequeue the front
        copyQ.pop();
    }

    // add a new line
    std::cout << std::endl;
}
```

Output of the code	
Analysis	In the main driver code, I successfully implemented queues using the STL library on the top and performed a queue operation showing how to push and pop characters in the queue list and also having a display feature to print the queue to the terminal. And for the output, it prints the implementation of queues inside the queue list including the character “K, R, & B”, and the three of it are pushed starting from K and ending with B, after pushing, the queue shows that K is the front and the B is the back of the list. After that, the whole set of the characters in the queue are displayed and dequeued it one by one to show the dequeue function and checks if the queue is empty and after that, the elements are all cleared.

ILO B.1

Task Title & Task Objective

Linked List Header of Queue

A 5.1 > ILO B > queue_ll.h > new_node<T>(T)

```
#ifndef QUEUELL_H
#define QUEUELL_H

// create a class node
template <typename T>
class qNode
{
public:
    T data;
    qNode *next;
};

// creating the new node
template <typename T>
qNode<T> *new_node(T newData)
{

    // allocate a space for the new node
    qNode<T> *newNode = new qNode<T>;
    // store the newData to the newNode
    newNode->data = newData;
    // point the newNode next to null
    newNode->next = nullptr;
    return newNode;
}

// enqueue
template <typename T>
void enqueue(T newData, qNode<T> **frontPtr, qNode<T> **backPtr)
{

    // create a new node
    qNode<T> *newNode = newNode(newData);

    // inserting into an empty queue
    //  how do we know if the queue is empty?
    //  check if the queue is empty
    if ((*frontPtr) == nullptr && (*backPtr) == nullptr)
```

```
void enqueue(T newData, qNode<T> **frontPtr, qNode<T> **backPtr)
    if ((*frontPtr) == nullptr && (*backPtr) == nullptr)

        // assign the frontPtr to the newNode
        (*frontPtr) = newNode;

        // assign the backPtr to the newNode
        (*backPtr) = newNode;
        return;
    }
    // if not then insert at the back
    // point the back next to the newNode
    (*backPtr)->next = newNode; // LINKS THE CURRENT BACK TO THE NEW NODE
    // the newNode will be the new backPtr
    (*backPtr) = newNode; // ASSIGNS THE BACKPTR TO THE NEW NODE
}

// dequeue
template <typename T>
void dequeue(qNode<T> **frontPtr, qNode<T> **backPtr)
{

    // check if the queue is empty
    if ((*frontPtr) == nullptr && (*backPtr) == nullptr)
    {
        std::cout << "The queue is empty. Nothing to dequeue" << std::endl;
        return;
    }
    // create a temporary variable to store the node to be deleted
    // creating a temporaty pointer
    qNode<T> *tempPtr = nullptr;

    // store the front pointer to the temporary pointer
    // check if the queue has only 1 item
    tempPtr = (*frontPtr);
```

```
    // check if the queue is 1 node left
    if ((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr)
    {

        // return only 1 item is inside the queue
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete tempPtr;
        return;
    }

    // update the frontPtr
    (*frontPtr) = (*frontPtr)->next;

    // point the tempPtr->next to null
    tempPtr->next = nullptr;

    // delete the node
    delete tempPtr;
}

// display all
template <typename T>
void display(qNode<T> *frontPtr)
{

    // catch if the queue is empty
    if (frontPtr == nullptr)
    {
        std::cout << "The queue is empty.\n";
        return;
    }

    //  temp pointer to store the front Ptr
```

```cpp
void display(qNode<T> *frontPtr)

    // catch if the queue is empty
    if (frontPtr == nullptr)
    {
        std::cout << "The queue is empty.\n";
        return;
    }

    //  temp pointer to store the front Ptr
    qNode<T> *temp = frontPtr;

    // create a loop that will print the tempPtr->data
    while (temp != nullptr)
    {
        std::cout << temp->data << " ";
        temp = temp->next;
    }

    std::cout << "\n";
}

// isEmpty
template <typename T>
bool isEmpty(qNode<T> **frontPtr, qNode<T> **backPtr) {
    if (*frontPtr == nullptr && *backPtr == nullptr) {
        std::cout << "The queue is empty" << std::endl;
        return true;
    }
    return false;
}
#endif
```

Main Driver Calling the Linked List in Queue With no STL	
Output of the code	
Analysis	

```
#include <iostream>
#include "queue_ll.h"
#include "queueArr.h"

int main() {

    // queue linked list
    qNode<int> *front = nullptr;
    qNode<int> *back = nullptr;

    enqueue(1, &front, &back);
    std::cout << "front: " << front->data << " back: " << back->data << "\n";
    enqueue(2, &front, &back);
    enqueue(3, &front, &back);
    std::cout << "front: " << front->data << " back: " << back->data << "\n";

    display(front);
    dequeue(&front, &back);
    std::cout << "front: " << front->data << " back: " << back->data << "\n";
    dequeue(&front, &back);
    std::cout << "front: " << front->data << " back: " << back->data << "\n";
    dequeue(&front, &back);
    dequeue(&front, &back);
    display(front);
```

```
Front: 1 back: 3
Front: 1 back: 4
Front: 1 back: 5
The Queue is full.
Front: 1 back: 5
The Queue is full.
Front: 1 back: 5
The Queue is empty.
The Queue is empty.
PS C:\Users\solvendar\OneDrive\Dokumen\CPE21S4_BAL>
```

ILO B.2

Task Title & Task Objective

Array Implementation Header of Queue

```
5.1 > ILO b > C queueArr.h > Dequeue()
#ifndef QUEUEARR_H
#define QUEUEARR_H
#include <iostream>

// create a class queue
template <typename T>
class queueArr {
    private:
    // pointer to the dynamically allocated arrau
    T *q_array;
    size_t q_capacity;
    size_t q_size;
    int q_front;
    int q_back;

public:

    // constructor
    queueArr(size_t capacity = 10);

        bool Empty();
        size_t Size();
        void Clear();

        T Front();
        T Back();

        void Enqueue(T value);
        void Dequeue();

        // copy constructor
        queueArr(const queueArr &other);

        // copy assignment operator
        queueArr &operator = (const queueArr &other);

        // destructor
        ~queueArr();

};

// constructor
template <typename T>
queueArr<T>::queueArr(size_t capacity) {
    q_capacity = capacity;
    q_size = 0;
    q_front = 0;
    q_back = -1;
    q_array = new T[q_capacity];
}
```

```cpp
template <typename T>
size_t queueArr<T>::Size() {
    return q_size;
}

template <typename T>
void queueArr<T>::Clear() {
    q_size = 0;
    q_front = 0;
    q_back = -1;
}

// front
template <typename T>
T queueArr<T>::Front() {
    if (Empty()) {
    std::cout << "The Queue is empty";
    return T{};
    }
    return q_array[q_front];
}

// back
template <typename T>
T queueArr<T>::Back() {
    if (Empty()) {
    std::cout << "The Queue is empty.\n";
    return T{};
    }
    return q_array[q_back];
}

template <typename T>
void queueArr<T>::Enqueue(T value) {

    // check if the enqueue is full
    if (q_size == q_capacity) {
        std::cout << "The Queue is full.\n";
        return;
    }
    // compute for the q_back for circular (use the modulo operator)
    q_back = (q_back + 1) % q_capacity;

    // assign the q_array with the updated q_back to the value
    q_array[q_back] = value;

    // increment the q_size
    q_size++;
}
```

```
// dequeue
template <typename T>
void queueArr<T>::Dequeue() {
    // check if the dequeue is empty
    if (Empty()) {
        std::cout << "The Queue is empty.\n";
        return;
    }
    // move q_front circularly (use the modulo operator)
    q_front = (q_front + 1) % q_capacity;

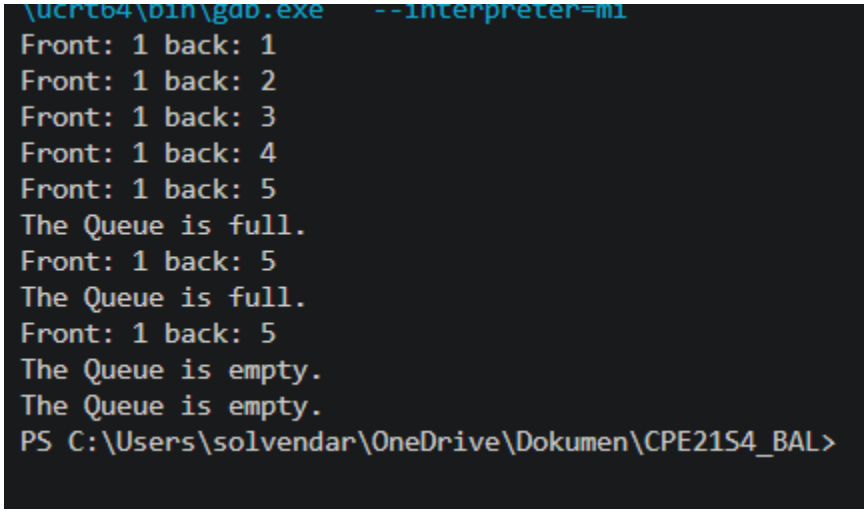
    // decrement the q_size
    q_size--;

    // reset index when queue is empty
    if (Empty()) {
        Clear();
        return;
    }
}

// destructor / delete
template <typename T>
queueArr<T>::~queueArr() {
    delete[] q_array;
}
#endif
```

Main Driver Calling the Array Implementation in Queue With no STL

```
    // object creation
    queueArr<int> q(5);
    q.Enqueue(1);
    std::cout << "Front: " << q.Front() << " back: " << q.Back() << "\n";
    q.Enqueue(2);
    std::cout << "Front: " << q.Front() << " back: " << q.Back() << "\n";
    q.Enqueue(3);
    std::cout << "Front: " << q.Front() << " back: " << q.Back() << "\n";
    q.Enqueue(4);
    std::cout << "Front: " << q.Front() << " back: " << q.Back() << "\n";
    q.Enqueue(5);
    std::cout << "Front: " << q.Front() << " back: " << q.Back() << "\n";
    q.Enqueue(6);
    std::cout << "Front: " << q.Front() << " back: " << q.Back() << "\n";
    q.Enqueue(7);
    std::cout << "Front: " << q.Front() << " back: " << q.Back() << "\n";
    q.Dequeue();
    q.Dequeue();
    q.Dequeue();
    q.Dequeue();
    q.Dequeue();
    q.Dequeue();
    q.Dequeue();
    return 0;
}
```

Output of the code	
Analysis	This stack array implementation header of queues is different from the linked list implementation. Containing class with private and public variables and the functions like constructor, copy constructor and destructor and the operations for the queue. The constructor helps to set the variables and the array will be the will used by the other functions to store the values and the copy constructor copy the values of the other variable to its own set of variables. While the copy assignment operator, copy the values and create an array and a new set of variables to its own. The terminal output displays the queue operations performed in the driver code, which have values of 1, 2, 3, 4, and 5, similar to the previous output. The difference is that the queue size is now displayed along with the total number of elements in the queue. To ensure that the queue size matches, there is a plus one in the front and back variables of the enqueue and dequeue functions. When the maximum value of 5 is reached, the same enqueue function returns a message indicating that the queue is full. The clear function then removes all of the elements, and the empty function determines whether the queue is empty.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

SUPPLEMENTARY:
ILO C

COMPUTER ENGINEERING

QUEZON CITY

Task Title & Objective:

Array Implementation Header

oA 5.1 > ILO C > C queueArr.h > ...

```h
#ifndef QUEUEARR_H
#define QUEUEARR_H
#include <iostream>
#include <string>

class Job
{

public:
    int ID;
    std::string username;
    int number_of_pages;
};

class Printer
{

private:
    // pointer to the dynamically allocated array
    Job *q_array;
    size_t q_capacity;
    size_t q_size;
    int q_front;
    int q_back;

public:
    // constructor
    Printer(size_t capacity = 10);

    // functions / methods that will be called out below
    bool isEmpty() const;
    void Clear();
    bool isFull() const;

    void addJob(Job value);
    Job RemoveJob();
    void DisplayJob();
    void JobProcess();

    // destructor
    ~Printer();
};

// creating class queue & importing the printer class to the constructor above
//  constructor
Printer::Printer(size_t capacity)
{
    q_capacity = capacity;
    q_size = 0;
    q_front = 0;
    q_back = -1;
```

```
51      q_back = -1;
52      q_array = new Job[q_capacity];
53  }
54
55  bool Printer::isFull() const
56  {
57      return q_size == q_capacity;
58  }
59
60  bool Printer::isEmpty() const
61  {
62      return q_size == 0;
63  }
64
65  void Printer::Clear()
66  {
67      q_size = 0;
68      q_front = 0;
69      q_back = -1;
70  }
71
72  void Printer::addJob(Job value)
73  {
74
75      // check if the Job is full
76      if (isFull())
77      {
78          std::cout << "The Queue of the Printer is full.\n";
79          return;
80      }
81      // else, use circular to compute the add for job
82      // compute for the q_back for circular
83      q_back = (q_back + 1) % q_capacity;
84
85      // assign the q_array with the updated q_back to the value
86      q_array[q_back] = value;
87
88      // increment the q_size
89      q_size++;
90
91      std::cout << "Job had been added.\n";
92  }
93
94  Job Printer::RemoveJob()
95  {
96
97      // check if the Job is empty
98      if (isEmpty())
```

```
    if (isEmpty())
    {
        std::cout << "No Job to print\n";
        return Job();
    }

    Job tail = q_array[q_front];
    // move q_front circularly
    q_front = (q_front + 1) % q_capacity;

    // decrement the q_size
    q_size--;

    // reset index when Job is empty
    if (isEmpty())
    {
        Clear();
    }
    return tail;
}

void Printer::DisplayJob()
{
    if (isEmpty())
    {
        std::cout << "There are no pending to print for jobs.\n";
        return;
    }
    // else, for loop to display the jobs
    int size = q_front;
    // displaying jobs using loop
    for (size_t i = 0; i < q_size; i++)
    {
        std::cout << "| ID: " << q_array[size].ID << "| Username: "
                  << q_array[size].username << "| Pages: " << q_array[size].number_of_pages << "|" << std::endl;

        size = (size + 1) % q_capacity;
    }
}
// destructor / delete
Printer::~Printer()
{
    delete[] q_array;
}

void Printer::JobProcess()
{
    if (isEmpty())
    {
```

```
void Printer::JobProcess()
{
    if (isEmpty())
    {
        std::cout << "No Job to process for now.\n";
        return;
    }
    // else, processing all jobs
    while (!isEmpty())
    {
        // to actively fetch jobs out of the array and to stop infinite loop freeze by reducing the queue size on every iteration
        Job tail = RemoveJob();
        std::cout << "| Printing Job ID: " << tail.ID << "| Username: " << tail.username << "| Pages: "
                  << tail.number_of_pages << "|" << std::endl;
    }
    std::cout << "The print for the job/s are done.\n";
}
#endif
```

Analysis

This header file works by setting up a system to manage print jobs. At the very top, the Job class acts as a simple container that holds three pieces of info, the job ID number, the username, and the total number of pages to print. By making these variables public, the printer class can easily look at and change this information without any blocks. Directly below that, the Printer class handles the queue logic using an array. Inside its private section, the printer tracks its status using five variables. It creates a pointer called q_array to hold the actual list of jobs, and it uses q_capacity and q_size to limit and count how many items are waiting. To make the array act like a continuous loop, it uses two index counters called q_front and q_back. The q_front

	variable points to the oldest job that needs to be printed next, while q_back tracks where the newest job was just placed. The addJob checks if the printer is full to stop early, but if there is space, it uses looping to move the back pointer and save the new job. Then, it adds one to the total size count and prints a message saying the job was added. The RemoveJob function checks if the queue is empty to avoid a crash, but if there are jobs waiting, it makes a quick copy of the data at the front pointer. After that, it moves the front pointer forward using the same looping, subtracts one from the total size, and gives the job back to the program. Lastly, the JobProcess function stops right away if the printer is empty, but if there are jobs, it starts a loop that runs until everything is cleared out. Inside this loop, it calls the remove function to pull files out one by one and prints their details to show they are printing.

Main Driver

```cpp
#include <iostream>
#include <string>
#include "queueArr.h"

int main() {

    Printer printer(10);
    int choice;
    Job tail; // declaring the "tail" so it can be used in case in switch

    do
    {

        std::cout << "\n==========================\n";
        std::cout << "          PRINTER\n";
        std::cout << "==========================\n";
        std::cout << "1. Add Print for Job\n";
        std::cout << "2. Process all Job\n";
        std::cout << "3. Remove Print Job\n";
        std::cout << "4. Display Print Job\n";
        std::cout << "5. Clear All\n";
        std::cout << "6. Exit\n"
                  << std::endl;
        std::cout << " Input your choice: ";
        std::cin >> choice;

        switch (choice)
        {
        case 1:
            std::cout << "Job ID:";
            std::cin >> tail.ID;
            std::cout << "Username: ";
            std::cin >> tail.username;
            std::cout << "Pages: ";
            std::cin >> tail.number_of_pages;

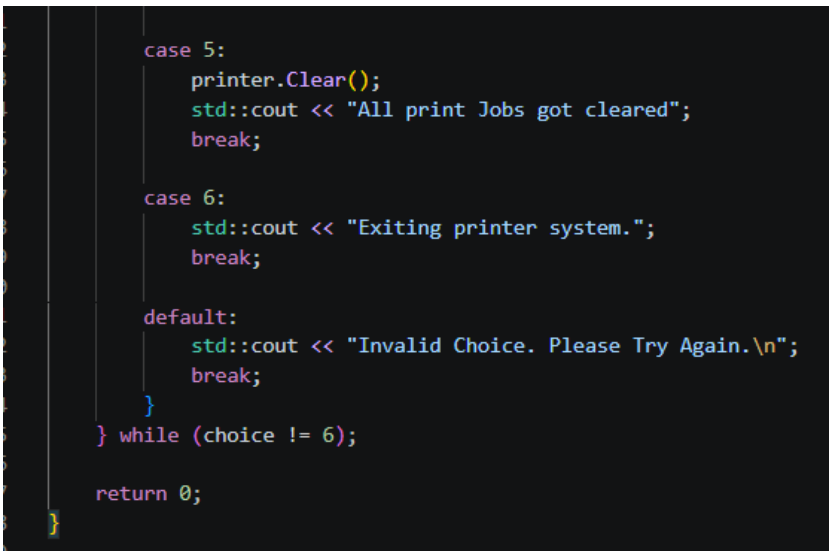
            printer.addJob(tail);
            break;

        case 2:
            printer.JobProcess();
            break;

        case 3:
            tail = printer.RemoveJob();
            break;

        case 4:
            printer.DisplayJob();
            break;

```

	
Analysis	In my main driver code, I created a printer object with a size of 10 and declared an int choice for my switch case and included the Job tail in the header file so I can implement it and use it inside the addJob function, while the rest was implemented in each cases and add a text after the function was called depends on the user's choice.
Output of the code	**Displaying the added Job:**

```
        case 5:
            printer.Clear();
            std::cout << "All print Jobs got cleared";
            break;

        case 6:
            std::cout << "Exiting printer system.";
            break;

        default:
            std::cout << "Invalid Choice. Please Try Again.\n";
            break;
        }
    } while (choice != 6);

    return 0;
}
```

```
============================
          PRINTER
============================
1. Add Print for Job
2. Process all Job
3. Remove Print Job
4. Display Print Job
5. Clear All
6. Exit

 Input your choice: 1
Job ID: 123
Username: Xen1
Pages: 12
Job had been added.

============================
          PRINTER
============================
1. Add Print for Job
2. Process all Job
3. Remove Print Job
4. Display Print Job
5. Clear All
6. Exit

 Input your choice: 1
Job ID: 456
Username: Xen2
Pages: 14
Job had been added.
```

```
============================
          PRINTER
============================
1. Add Print for Job
2. Process all Job
3. Remove Print Job
4. Display Print Job
5. Clear All
6. Exit

 Input your choice: 1
Job ID: 7890
Username: Xen3
Pages: 15
Job had been added.
```

Displaying the Job had been added:

```
============================
          PRINTER
============================
1. Add Print for Job
2. Process all Job
3. Remove Print Job
4. Display Print Job
5. Clear All
6. Exit

 Input your choice: 4
| ID: 123| Username: Xen1| Pages: 12|
| ID: 456| Username: Xen2| Pages: 14|
| ID: 7890| Username: Xen3| Pages: 15|
```

Else:

```
============================
          PRINTER
============================
1. Add Print for Job
2. Process all Job
3. Remove Print Job
4. Display Print Job
5. Clear All
6. Exit

 Input your choice: 4
There are no pending to print for jobs.
```

Removing Print Job Showing the FIFO:

```
============================
          PRINTER
============================
1. Add Print for Job
2. Process all Job
3. Remove Print Job
4. Display Print Job
5. Clear All
6. Exit

 Input your choice: 3

============================
          PRINTER
============================
1. Add Print for Job
2. Process all Job
3. Remove Print Job
4. Display Print Job
5. Clear All
6. Exit

 Input your choice: 4
| ID: 456| Username: Xen2| Pages: 14|
| ID: 7890| Username: Xen3| Pages: 15|
```

Process Job:

```
============================
          PRINTER
============================
1. Add Print for Job
2. Process all Job
3. Remove Print Job
4. Display Print Job
5. Clear All
6. Exit

 Input your choice: 2
| Printing Job ID: 456| Username: Xen2| Pages: 14|
| Printing Job ID: 7890| Username: Xen3| Pages: 15|
The print for the job/s are done.
```

Else:

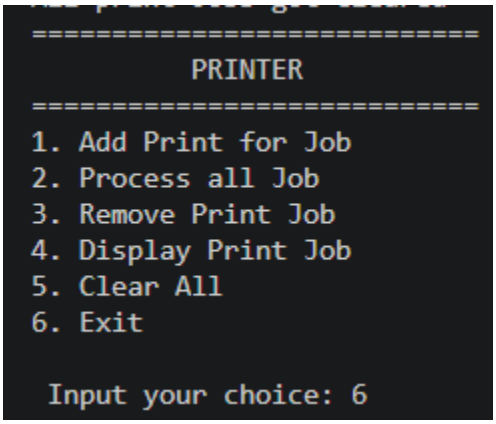
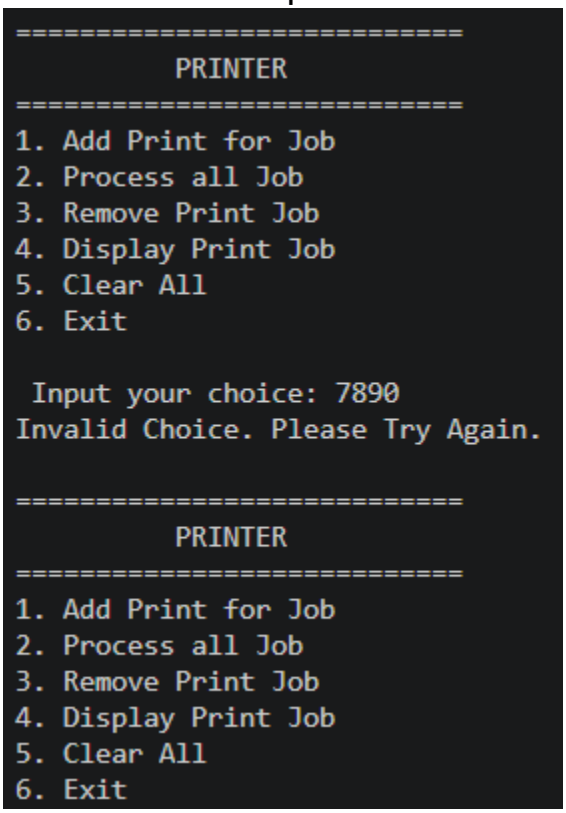
```
============================
          PRINTER
============================
1. Add Print for Job
2. Process all Job
3. Remove Print Job
4. Display Print Job
5. Clear All
6. Exit

 Input your choice: 2
No Job to process for now.
```

Clearing All:

```
============================
          PRINTER
============================
1. Add Print for Job
2. Process all Job
3. Remove Print Job
4. Display Print Job
5. Clear All
6. Exit

 Input your choice: 5
All print Jobs got cleared
```

	Exit:  **If none of the choice is equal to 6:** 
Analysis	This was the output of the program inside the terminal that contains the user input in the add job function that called enqueue in STL queue and also remove job that called dequeue, Display Print Job that showing the added job to the printer that shows successfully the queue without the STL library of it and using array implementation header to manage to perform addJob that let input the ID, username, and the number of pages, removeJob showing the

	FIFO inside of it including the processJob, and display the printed jobs.
Why did you use Array Implementation instead of Linked List Implementation?	Using a circular array makes it much safer to run my printer program because the maximum number of print jobs can be specified directly in the code. Because my print jobs are FIFO, using an array uses memory more efficiently, resulting in less time and less memory usage. It is also much easier for me to visualize because the data appears as a clean list of values and addresses arranged in a row. However, linked lists are more confusing to me because they involve separate items connected in a line. Finally, using an array made it easier to see how elements are stored in arranged memory locations without requiring additional pointer references.

Conclusion
In summary, this practical activity effectively illustrated the concepts of queues by letting me build a printer program without using the STL library. By creating my own custom header file, I successfully set up the main structure of the system using a circular array to handle incoming print requests. The tasks showed how a printer directly uses standard queue operations, such as enqueue through my addJob function, dequeue through my RemoveJob function, and the two check functions, isEmpty and isFull. Using a circular array inside my code was effective because it let me set a strict limit of 10 print jobs, which uses memory efficiently and keeps things running fast. By running these algorithms in a shared printer scenario, I learned how to create reusable code and being organized.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explaining the concepts, Idea Generation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Explaining the concepts about having a 2 class inside the array implementation and created a code to visualize as an example and not align to the supplementary because the code that i used was the B.2 and just remodify it.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or

completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.